

计算导论与程序设计

概念-期末回顾



软件与理论中心

张艳梅

计算的定义

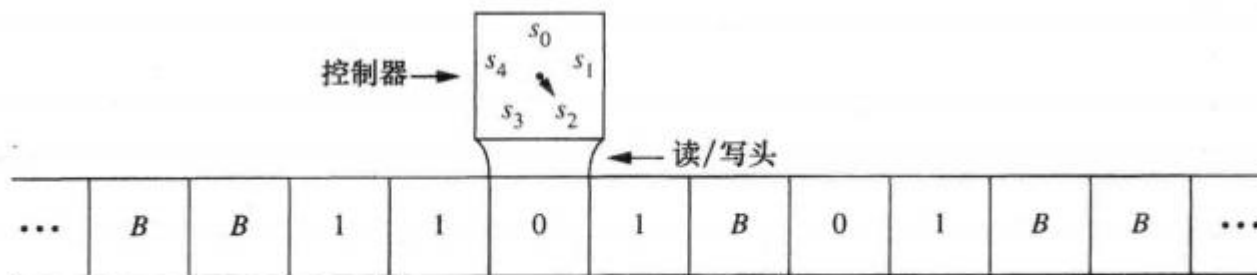
- 以数字为基础、遵循一定的**计算规则**进行；
如：数的加减乘除，函数的微分、积分、方程的求解、定理的证明推导；
- 广义：计算就是把一个符号串 f 变换成另一个符号串 g ；
- 更广义：计算就是对信息的变换；
- 由于计算规则的机械化、公式化，可以借助计算工具来实现计算。

图灵机是计算机的理论模型

- 图灵机的定义: $T=(S, I, f, s_0)$

由四部分组成, 有限状态集 S , 包含空白符 B 的字母表 I , 从 $S \times I$ 到 $S \times I \times \{R, L\}$ 的部分函数 f , 以及初始状态 s_0 。

$f: (s, x) \rightarrow (s', x', d)$ 转移规则



带在两个方向都是无限的
任何时刻仅有有限多个非空白小方格

图 1 图灵机的表示

巴贝奇分析机-计算机的定义

1834年巴贝奇提出**分析机**构想:一台通用的数学计算机，不仅可以进行数字运算，而且还能进行逻辑运算（进行逻辑判断的非数值运算）。

巴贝奇的分析机由四部分构成：

第一部分：保存数据的齿轮式“**存贮仓库**”(Store)；

第二部分：对数据进行各种运算的装置(“**作坊**”)；

第三部分：对操作顺序进行**控制**、并对所要处理的数据及输出结果加以选择的装置；

第四部分：送入和取出数据、以及在“存贮仓库”和“作坊”之间运输数据的**输入输出部件**。

冯.诺依曼机体系结构的特点：

- 指令和数据采用二进制表示，从而简化机器的逻辑线路；——指令和数据的表示
- 指令和数据一样存储在主存储器中；——指令和数据的存储
- 计算机由运算器、控制器、存储器、输入设备、输出设备五大部分组成。

第一台基于冯.诺伊曼思想的计算机于1950年在美国宾西法尼亚大学诞生，名为EDVAC。



英国数学家布尔
G. Boole

1847年布尔出版了
《逻辑的数学分析》

1854年出版《思维规律
的研究——逻辑与概率
的数学理论基础》

- 布尔构思出一个关于0和1的代数系统，用基础的逻辑符号系统描述物体和概念，将逻辑推理过程简化为极为容易和简单的代数运算。
- 在布尔代数中，0和1是自变量的定义域，也是函数的值域。基本运算只有与、或、非 三种。



仙农和开关电路理论

1938年发表了《继电器和开关电路的符号分析》

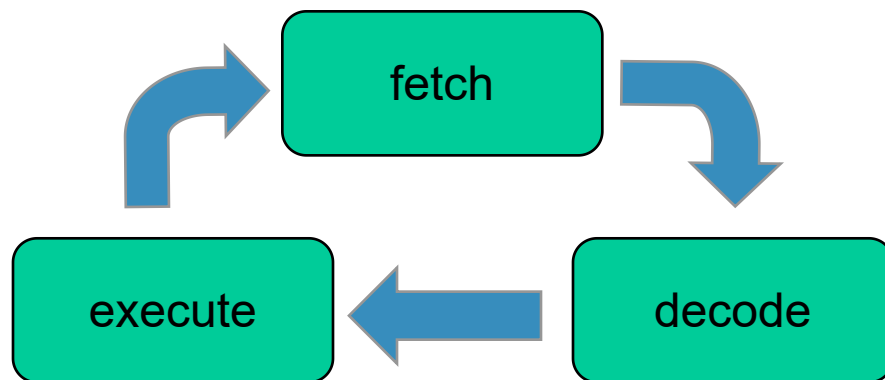
- 首次以布尔代数作为工具进行开关电路分析、设计。
- 证明可以通过继电器电路来实现布尔代数的逻辑运算。
- 明确地给出了实现加、减、乘、除等运算的电子电路的设计方法。

后来，进一步证明了可以采用能实现**布尔代数运算**的继电器或电子元件来制造计算机，使计算机具有算术运算和逻辑运算功能。

- 信号是离散的
 - 通过对模拟信息取样来实现Sampling is only one way to digitize information
 - 每件事物都必须变成数字！
 - 文字可以用ASCII、Unicode等进行数字编码。Text (letters and special characters) gets converted to numbers ($A = 65$), using a standard coding convention called **ASCII**
 - 图片要切割为像素和颜色数值。Graphics (images), gets broken down into pieces (pixels) and each colour gets a number

- 一幅位图图像可以看作是由多个小方格组合而成的矩阵，每个小方格叫做一个"像素"，比如一个分辨率为 1024×768 的图像可看作由 1024×768 个方格组成。假设每个像素能表示512种颜色，则存储一幅 1024×768 的图像至少需要多少字节？请说明原因。
- 每像素需要 $\log_2 512 = 9\text{bit}$, 9/8 向上取整=2字节。
- 存 1024×768 分辨率图像至少需要 $2 \times 1024 \times 768 = 1536 \times 1024$ 字节 = 1536 KB

程序的执行过程



程序的运行就是不断地 取指令、分析指令、执行指令的过程，这3个步骤均由控制器来控制。控制器取指令、分析指令，产生操作控制信号发给输入输出设备、运算部件或者主存，完成指令的执行。

一般计算机包括如下几类指令：

- 1) **算术运算类**。执行加、减、乘、除等算术运算的指令类；
- 2) **逻辑运算类**。执行与、或、非、移位、比较等逻辑运算的指令类；
- 3) **传送类**。执行取数、存数、传送等操作的指令类；
- 4) **程序控制类**。执行无条件转移、条件转移、调用程序、返回等操作的指令类；
- 5) **输入/输出类**。执行输入、输出、输入/输出等实现内存和外部设备之间传输信息操作的指令类；
- 6) **其他类指令**。执行停机、空操作、等待等操作的指令类。

指令的组成:

操作码

地址码

操作码: 用来表明本条指令要求计算机完成的操作;

操作数地址: CPU根据该地址取得所需的操作数; 可能直接给出操作数, 可能是内存地址, 也可能是寄存器地址 (即寄存器名);

操作结果的存储地址: 将对操作数的操作结果保存在该地址中, 以便再次使用; 可能是内存地址, 也可能是寄存器地址;

下一条指令的地址: 一般的, 如果程序是顺序执行, 则下一条指令的地址由程序计数器PC (存放下一条指令地址的寄存器) 指出; 仅当改变程序的运行顺序 (转移、调用子程序) 时, 下条指令的地址才由转移类指令给出。

操作码和地址码均是二进制数据

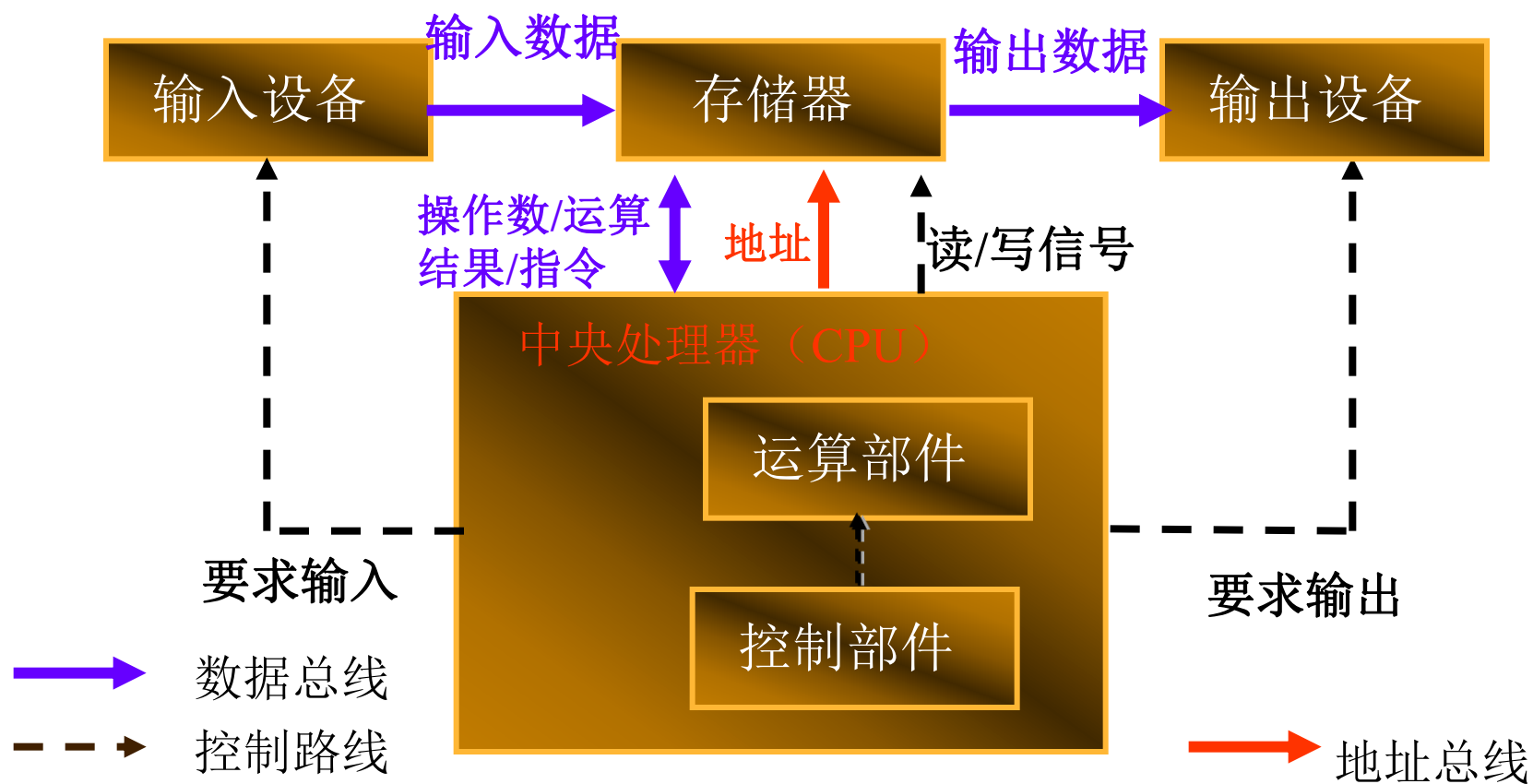
指令系统

计算机能直接识别和执行的全部指令的集合，称为该种计算机的指令系统。

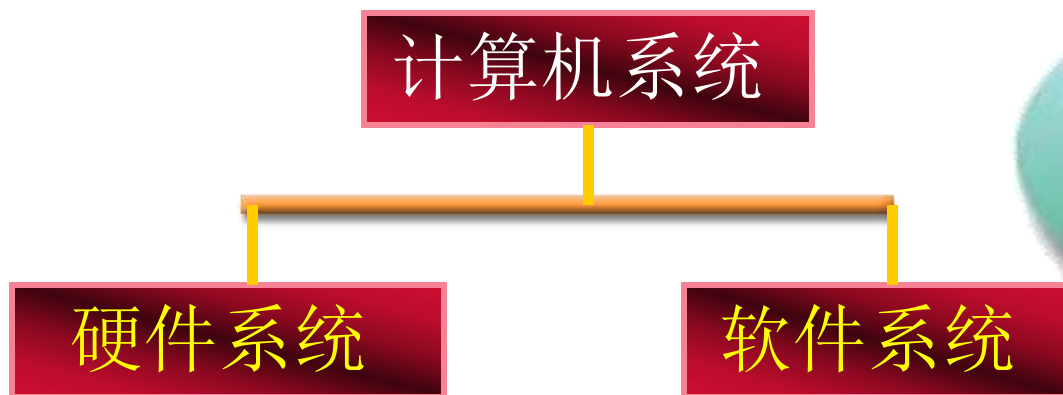
程序

程序是按事先设计的功能和性能要求编制的指令序列。

注意：这种提法并不都是指程序必须是二进制代码，而是指程序最终会转换为指令序列！



计算机系统的组成



计算机系统由硬件和软件两大部分组成。

硬件是计算机的物质基础；软件可以发挥和扩大计算机硬件功能，两者相辅相成，缺一不可

二、计算机软件及其分类

软件的定义：是程序、数据、相关文档的完整集合。

程序

按事先设计的功能和性能要求编制的指令序列

数据

程序能正常操纵的信息的数据结构

文档

与程序开发、维护和使用有关的图文材料

软件 ≠ 程序

系统软件

面对普遍的应用要求、为了充分发挥硬件的特性、维护计算机的硬件和软件系统、平滑用户与硬件设备间的沟坎而设计的软件，不针对具体的应用。

如：操作系统、数据库管理系统、设备驱动程序、语言集成开发环境（如Dev-C++）等。

应用软件

是在系统软件的基础上开发出来的、用于解决具体问题所编制的各种具体应用程序，如：飞机订票系统，图书管理系统等。

系统软件和应用软件的区分有时并不严格，没有必要死抠！

操作系统的定义：

操作系统是这样一些**程序**模块的集合——它们**管理和控制**计算机系统**中的硬件和软件资源**，**合理组织计算机工作流程**（调度运行用户应用程序），以便有效地利用这些资源为用户提供一个功能强大、使用方便和可扩展的工作环境，从而在计算机与其用户之间起到**接口**作用。

常见的操作系统有：MS-DOS（又称PC-DOS）、UCDOS、Windows、UNIX、LINUX等；

系统软件：操控、管理资源，提供接口隔离硬件

● 操作系统的五大功能

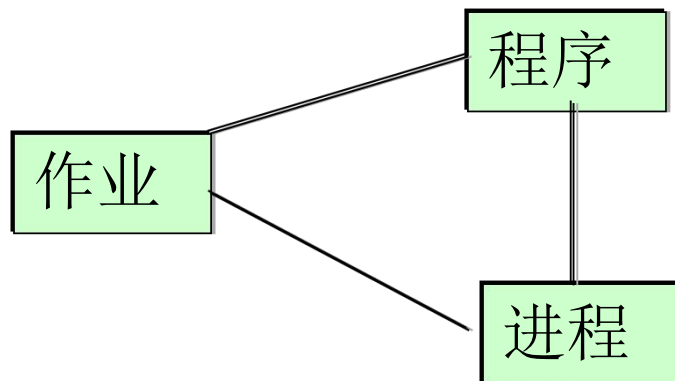
作业管理、CPU(进程)管理、内存管理、设备管理、文件管理

- 作业：用户请求计算机计算的一个计算任务，由程序、数据和作业说明书组成，用于完成用户的一个计算目标。
- 进程：是一个具有独立功能的程序对某个数据集在处理机上的执行过程，是资源分配的基本单位。一个程序（作业）的执行总是会产生一个或多个进程。

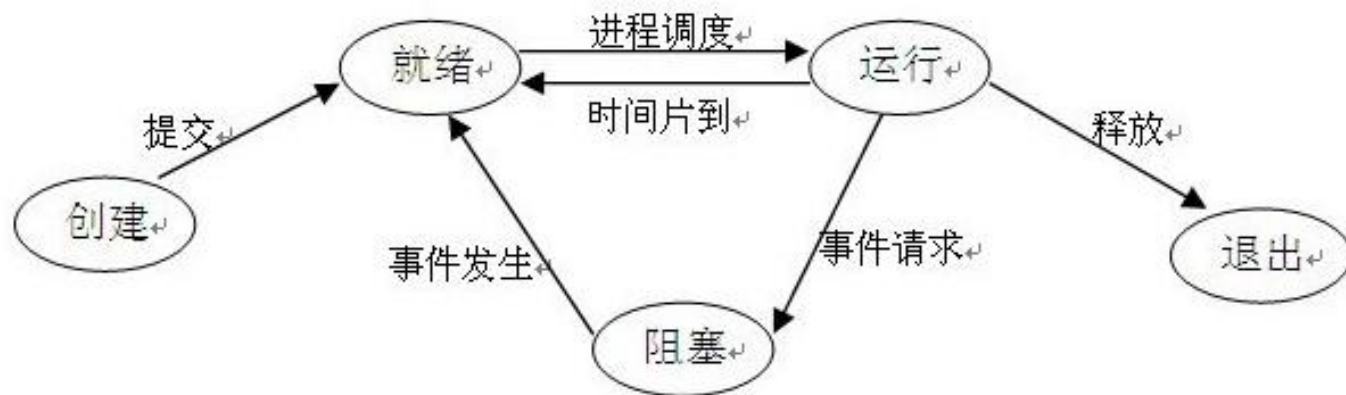
程序与进程的区别

- 程序与进程的关系

- 进程是动态的，强调执行过程，而程序是静态的；
- 进程具有并发性(宏观上同时运行)，程序没有；
- 不同的进程可以对应同一程序，只是该程序对应的数据集不同。



进程三态状态转换图



(1) 运行：当一个进程在处理机上运行时，则称该进程处于运行状态。处于此状态的进程的数目小于等于处理器的数目，对于单处理机系统，处于运行状态的进程只有一个。在没有其他进程可以执行时（如所有进程都在阻塞状态），通常会自动执行系统的空闲进程。

(2) 就绪：当一个进程获得了除处理机以外的一切所需资源，一旦得到处理机即可运行，则称此进程处于就绪状态。就绪进程可以按多个优先级来划分队列。例如，当一个进程由于时间片用完而进入就绪状态时，排入低优先级队列；当进程由I/O操作完成而进入就绪状态时，排入高优先级队列。

(3) 阻塞：也称为等待或睡眠状态，一个进程正在等待某一事件发生（例如请求I/O而等待I/O完成等）而暂时停止运行，这时即使把处理机分配给进程也无法运行，故称该进程处于阻塞状态。

定义: 算法是一个有限步骤的求解序列, 用来解决一类特定的问题。

例: 描述在一个有限整数序列 $\{a_n\}$ 中寻找最大值的算法。

解: 算法步骤如下:

1. 设置最大值 $\max$ 初始值, 为序列中第一个整数 a_1 .
2. ***for*** $i=2..n$
 if $(\max < a_i)$ *then* $\max = a_i$
3. *return* $\max \{ \max = \max(a_n) \}$

- **输入:** 算法应当有指定集的输入值.
- **输出:** 对应输入值, 算法应给出指定集的输出值, 即问题的解.
- **正确性:** 算法应针对每一组输入都能产生正确的输出.
- **有限性:** 算法应对任何输入值都能在有限步后给出结果.
- **可执行性:** 算法的每一步都能在有限时间内正确执行.
- **通用性:** 算法应可以适用既定问题的所有情况.

- 很多复杂问题，不能找到直接的求解公式；只能通过把问题归约到更小输入的问题实例来解决，或者说 $f(n)$ 的计算依赖于 $f(n-k)$ 或 $f(n/k)$ 的求解。
- 算法 $f(n)$ 的计算公式，依赖于较小规模的问题实例 $f(n-k)$ 或 $f(n/k)$ 的求解，这种计算公式叫做递归定义。
- 使用迭代算法编写程序的基本方法：
 - 寻找问题求解的递归公式；
 - 设置迭代求解的初始值；
 - 设计循环结构，实现一步步的从小到大的迭代计算。

- 递归函数算法：直接用递归定义设计一个递归函数，然后调用这个递归函数从 $f(n)$ 一步步递推至初始条件，再自动逐级函数回归完成计算，这样的过程称为递归(recurrsion)。
- 递归算法必须要有初始条件，不能无限递归下去。
- 递归函数的特点：
 - 必须有正确的递归定义；
 - 回归阶段的反序性，适合执行反向计算；
 - 克服正向推理难以控制的复杂度。

- 蛮力算法用最直接的方式求解问题, 一点不考虑如何使算法更有效.
- 蛮力算法的例子有穷举算法、线性搜索, 冒泡排序和插入排序等。
- 穷举法解题思路: 对所有可能是解的众多 候选解 按某种顺序进行 逐一列举 和逻辑检验, 从中找出那些符合要求的候选解作为问题的解。
- 穷举算法的设计核心:
 - 定义所有候选解的集合, 即“解空间”;

- 确定的有限自动机的形式化定义

确定的有限自动机是一个五元组

其中:

- : 有限的状态集合;
- : 有限的输入字母表;
- : 转换函数, 是 到 的映射;
- : 初始状态, ; (初始状态只有一个)
- : 终止状态集, ;

有限状态机：设计状态转换图

- 如何设计有限状态自动机？
- 可能出现的有限个状态、运行的规则都是事先确定的。一旦开始运行，就按照事先确定的规则工作，因此叫“自动机”。
- 使用状态转换图形直观描述整个工作过程。

有限状态机的通用编码模式

```
#define START 1  
  
...  
int state=START;  
  
...  
while (state!=END) {  
    ch=getchar();  
    switch(state) {  
        case START:  
            if (ch==?)  
                state=Q1; break;  
        ...  
    }  
}
```

有限状态机解题通用处理模式

1.1 什么是程序设计语言

一、语言的概念：

- 语言就广义而言，是一套共同采用的 沟通符号、表达方式与处理规则。
- **计算机语言 (Computer Language)** 指用于人与计算机之间通讯的语言。
- 程序设计语言介于人与人沟通的语言和机器语言之间。

1、机器语言

- ◆ 计算机发展早期使用的语言；面向计算机指令。
- ◆ 指令由“0”和“1”的二进制码组成，是指挥计算机工作的命令，是计算机唯一可以直接识别的语言；
- ◆ 与具体机器有关，不同的机器能识别的机器语言也不同；

1.2 程序设计语言的演化—符号语言

2、符号语言

- ◆ 20世纪50年代早期，数学家Grace Hopper发明了符号语言，即用符号或助记符来表示不同的机器语言指令（包括操作码和和操作数地址）。
- ◆ 程序员可用ADD、SUB、MUL、DIV等符号来分别表示加法、减法、乘法、除法的操作码。
- ◆ 符号语言又称汇编语言。

1.2 程序设计语言的演化—高级语言

3、高级语言

- ◆ 由于汇编语言的局限性，后来出现了高级语言。

高级语言的优势

- ◆ 高级语言程序易学、易懂、也易查错。
- ◆ 使程序员可以完全不用与计算机的硬件打交道、不必了解机器的指令系统。
- ◆ 高级语言与具体机器无关，在一种机器上运行的高级语言程序有可能可以不经改动地移植到另一种机器上运行，大大提高了程序的通用性。

1.2 程序设计语言的演化

4、第四代语言

第四代语言(Fourth-Generation Language, 以下简称4GL) 是比3GL更加靠近自然语言的编程语言。

旨在提供更高水平的内部计算机硬件细节的抽象, 使该语言更加对程序员友好、强大和通用。

它可以通过同时更多地操作大量的信息集合, 而不是仅仅关注比特和字节。可能支持数据库管理、报告生成、数学优化、gui开发或web开发。

1.2 程序设计语言的演化

5、自然语言

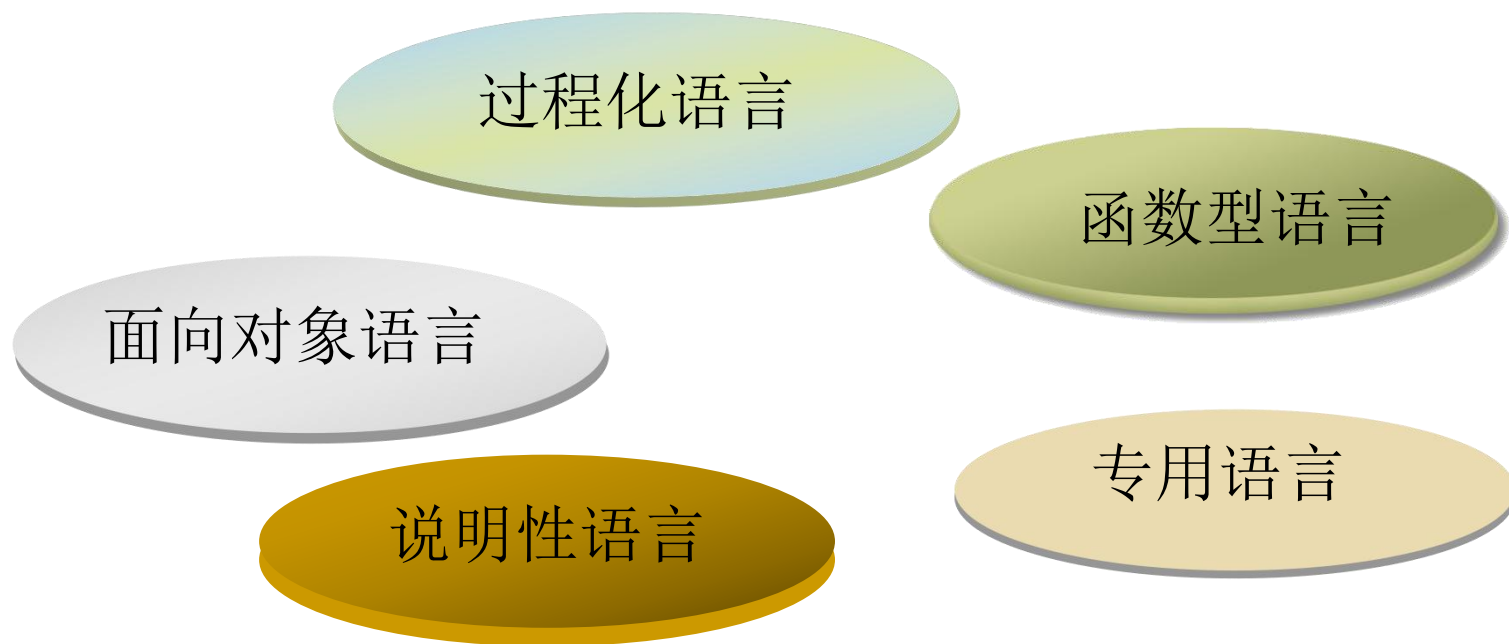
理想情况下，计算机能够理解自然语言（如英语、汉语等）并立即执行请求。

大量关于自然语言的工作正在实验室中进行。

但迄今为止，自然语言的使用仍然是相当有限的。

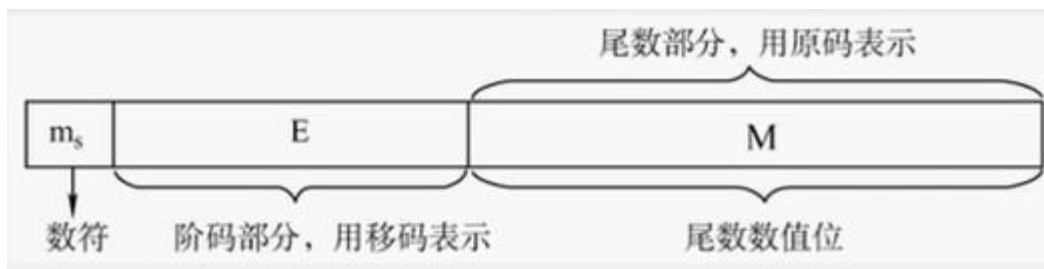
1.4 程序设计语言的分类

- 根据人们对于求解问题的方法进行分类可以将程序设计语言进行分类。

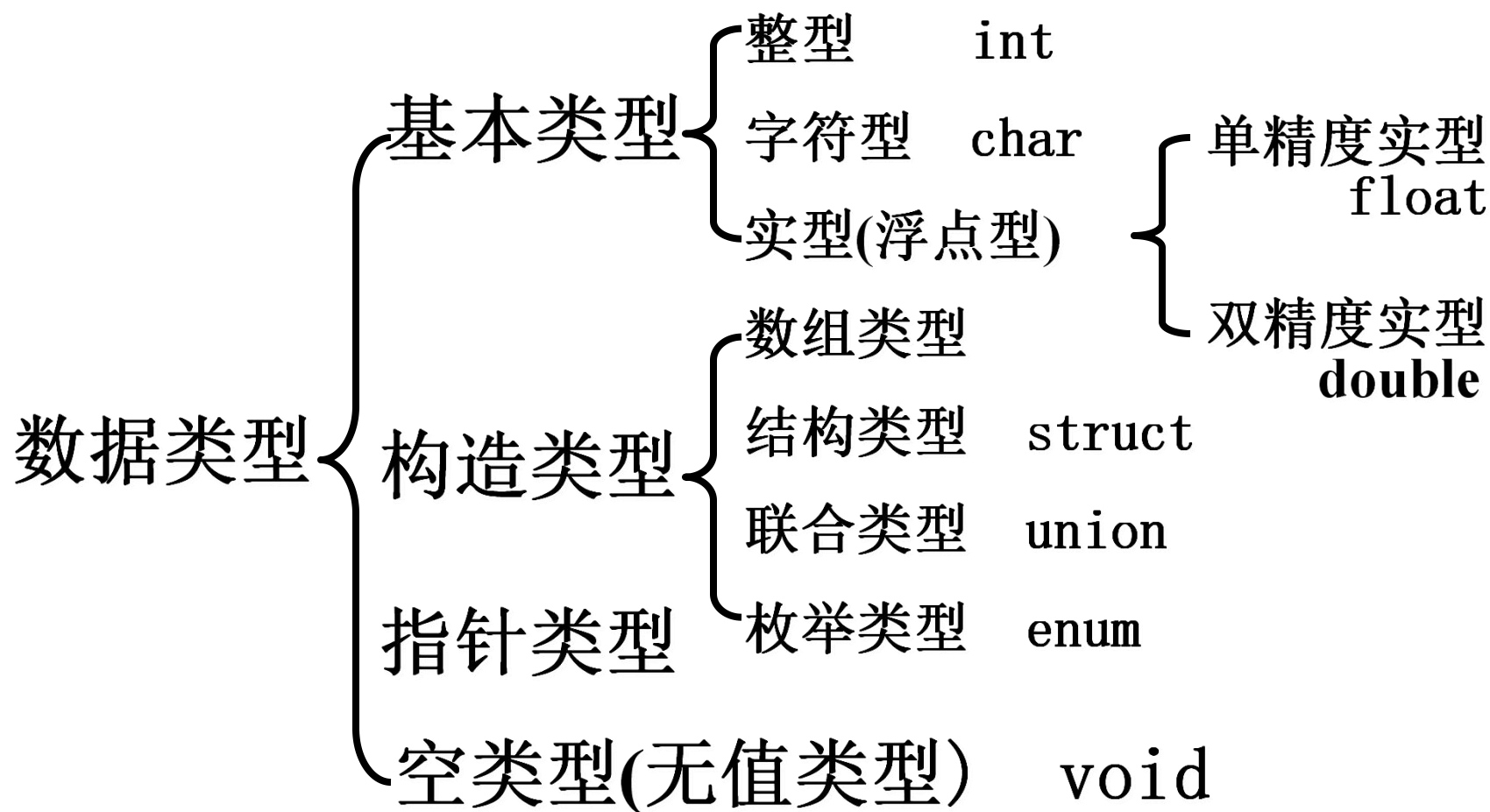


- 数据类型 包括三要素(含义):
 - 逻辑结构, 定义了数据的抽象/值的集合(属于该类型的数据能够**取值的范围**) ;
 - 存储结构, 规定了数据的存储方式;
 - 能应用于这些数值上的操作集合 (**数据操作**) 。
- 浮点数集合, 支持算术运算、数学函数运算。
- 整数集合, 支持字符编码转换, 以及整数的加、减、乘和整数除法运算。

- 计算机中存储一个数据的物理空间是固定的，即一个**字长**。
- 一台32位机的计算机上，整数108这样存储：
00000000 00000000 00000000 01101100
- 但是小数的计算机表示形式，采用"尾数"(这是一个纯小数)和"指数"(这是一个整数)组合表示。
例如： $1001.011 = 2^{3(011)} \times (1.001011)$



C语言提供了以下数据类型：



结构类型

- 结构是用其他类型的对象构造出来的派生数据类型。
- 声明一个结构体类型的一般形式为：

```
struct 结构体名 {  
    成员列表  
};
```

typedef 用于为经常出现的类型创建一个别名。

```
typedef struct Book Mybook;  
typedef struct {  
    double x; double y;  
} Rect;
```

结构变量的引用

- 不能整体访问结构变量，只能按成员访问

结构成员运算符“.”，通过结构变量名访问结构成员：**结构变量名.结构成员名**

例：scanf("%f", **&library.value**);

// .运算符优先级比&高

指针运算符“->”，通过指向结构变量的指针

访问结构成员：**指针名->结构成员名**

或 **(*指针名).结构成员名**

- 结构变量可以进行赋值运算。

枚举类型

枚举：将变量的值一一列举出来，变量的值只限于列举出来的值的范围内。

申明枚举类型用enum

```
enum weekday {sun, mon, tue, wed, thu, fri, sat};
```

定义变量：

```
enum weekday workday, week-day;  
enum {sun, mon, tue, wed, thu, fri, sat} workday;  
变量值只能是sun到sat之一。
```

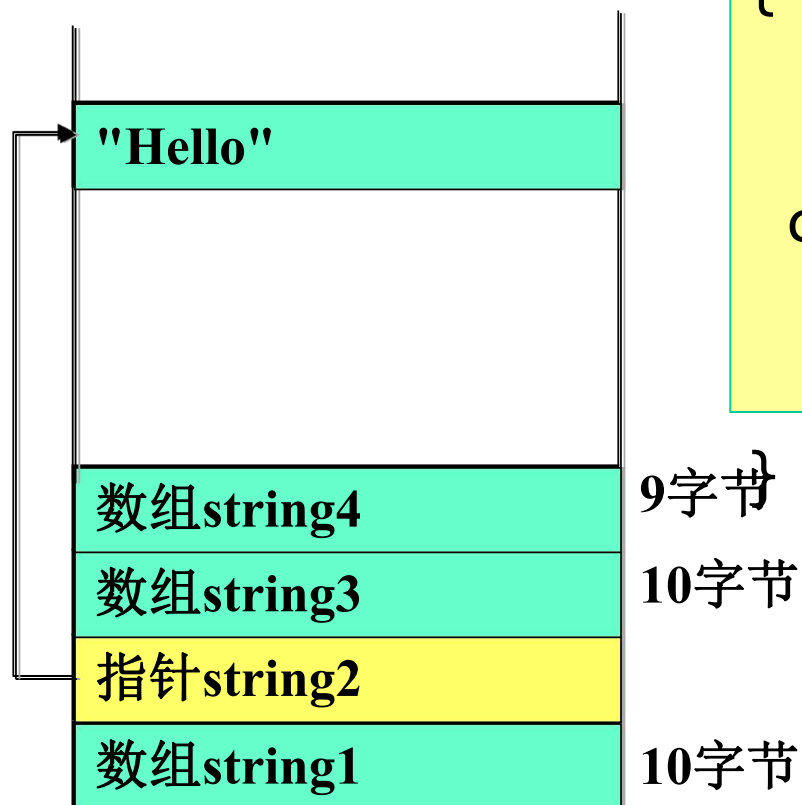
- 常量是指在程序执行过程中不能改变的数据。常量按表示形式分为：文字常量、命名常量、符号常量。
- **文字常量**是指在程序中直接出现的数据。
- **符号常量**：程序代码中预先统一设定的文字常量，并以自定义的标识符表示。例：`#define N 100`
- **不能改写的变量(命名常量)**

添加**const**修饰符的变量一旦声明并初始化，便不允许程序去改变该存储空间中的数据。

int * const iPtr; /*iPtr不能被修改，只能指向i*/

const int * iPtr; /*iPtr指向的整数变量不能被修改*/

在内存的常量区里保存"Hello"



```
int main()
{
    char string1[10],
        string3[10];
    const char * string2 = "Hello";
    char string4[] = "Good Bye";
    .....
```

8字节 (`sizeof(char *)`值为8) 指针的字节数和计算机字长一致.

- ◆ 表达式 (expression) 是由运算符、操作数和括号组成的、计算求值的基本单位。
- ◆ 操作数可以是：常量、变量、函数调用和表达式；
- ◆ 表达式举例：
 - ◆ 2 表达式无运算符，操作数为常量
 - ◆ i 表达式无运算符，操作数为变量
 - ◆ $\sin(x)$ 表达式无运算符，操作数为函数调用
 - ◆ $2 + i$ 操作数为常量和变量
 - ◆ $i * \cos(x)$ 操作数为变量和函数调用
 - ◆ $2 + i * \sin(x)$ 加法运算的第2个操作数仍为表达式

3.1 表达式的递归定义

表达式的定义是一个递归的形式，由下面四条规则构成：

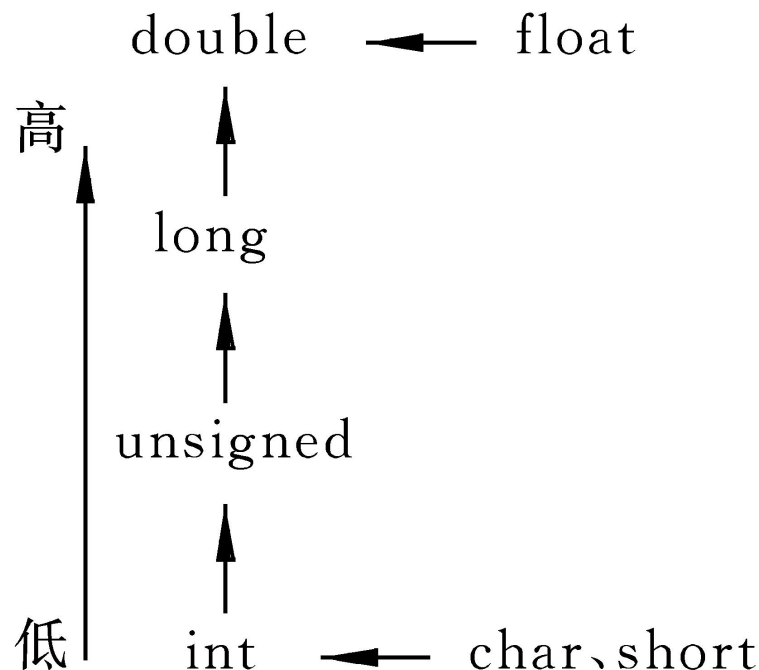
- (1) 常量、变量、函数调用是表达式。
- (2) 如果 Ω 是表达式，则 (Ω) 是表达式；如果 θ 是单目运算符， θ 允许前缀形式，则 $\theta\Omega$ 是表达式；如果 θ 允许后缀形式， $\Omega\theta$ 是表达式。
- (3) 如果 Ω_1 、 Ω_2 、 Ω_3 是表达式， θ 是双目运算符，则 $\Omega_1\theta\Omega_2$ 是表达式； θ_1 和 θ_2 构成三目运算符，则 $\Omega_1\theta_1\Omega_2\theta_2\Omega_3$ 是表达式。
- (4) 只有有限次的使用上述规则得到的有限长度的字符串式子才是表达式。

各类数值型数据间的混合运算

混合运算：整型(包括int, short, long)、浮点型(包括float, double)可以混合运算。在进行运算时, 不同类型的数据要先转换成同一类型, 然后进行运算.

说明:

这种类型转换是由系统自动进行的。



3.8.1 C 运算符简介

C 的运算符有以下几类：

- (1) 算术运算符 (+, -, *, /, %)
- (2) 关系运算符 (>, <, ==, >=, <=, !=)
- (3) 逻辑运算符 (!, &&, ||)
- (4) 位运算符 (<<, >>, ~, |, ^, &)
- (5) 赋值运算符 (=, +=, -=, *=, /=, %=)
- (6) 条件运算符 (? :)
- (7) 逗号运算符 (,)

(8) 指针运算符 (*和&)

(9) 求字节数运算符 (sizeof())

(10) 强制类型转换运算符 ((类型名))

(11) 分量运算符 (. , ->)

(12) 下标运算符 ([])

(13) 其他 (如函数调用)

运算符的优先级和结合性

- C 语言规定了运算符的优先级和结合性。
在表达式求值时,先按运算符的优先级别高低次序执行,例如先乘除后加减。
- C 规定了各种运算符的结合方向(结合性)
算术运算符的结合方向为“自左至右”,即先左后右。
 - $++i$, $--i$ (在使用 i 之前,先使 i 的值加(减) 1)
 - $i++$, $i--$ (在使用 i 之后,使 i 的值加(减) 1)
- ANSI C 并没有具体规定表达式中的子表达式的求值顺序,允许各编译系统自己安排。

写出下面各逻辑表达式的值。设 $a=3, b=4, c=5$ 。

(1) $a + b > c \ \&\& \ b == c$

(2) $a \parallel b + c \ \&\& \ b - c$

(3) $!(a > b) \ \&\& \ !c \parallel 1$

(4) $!(x = a) \ \&\& \ (y = b) \ \&\& \ 0$

(5) $!(a + b) + c - 1 \ \&\& \ b + c / 2$

解题思路：

关系运算符的优先级高于赋值运算符,但低于算术运算符;

逻辑运算符优先级: $!$ 高于 $\&\&$, $\&\&$ 高于 $\parallel$

有括号优先括号。

赋值表达式的嵌套特性

- ✓ 赋值表达式, 可以作为子表达式嵌入其他表达式中。例如: `while (ch=getchar) != '\n' ) ...`

表达式 “`a=(b=5)`” 分析: 括弧内的 “`b=5`” 是一个赋值表达式, 相当于执行

“`b=5`” 和 “`a=b`” 两个赋值表达式。

赋值运算符按照 “自右而左” 的结合顺序,

因此, “`(b=5)`” 外面的括弧可以不要, 即 “`a=(b=5)`” 和 “`a=b=5`” 等价。

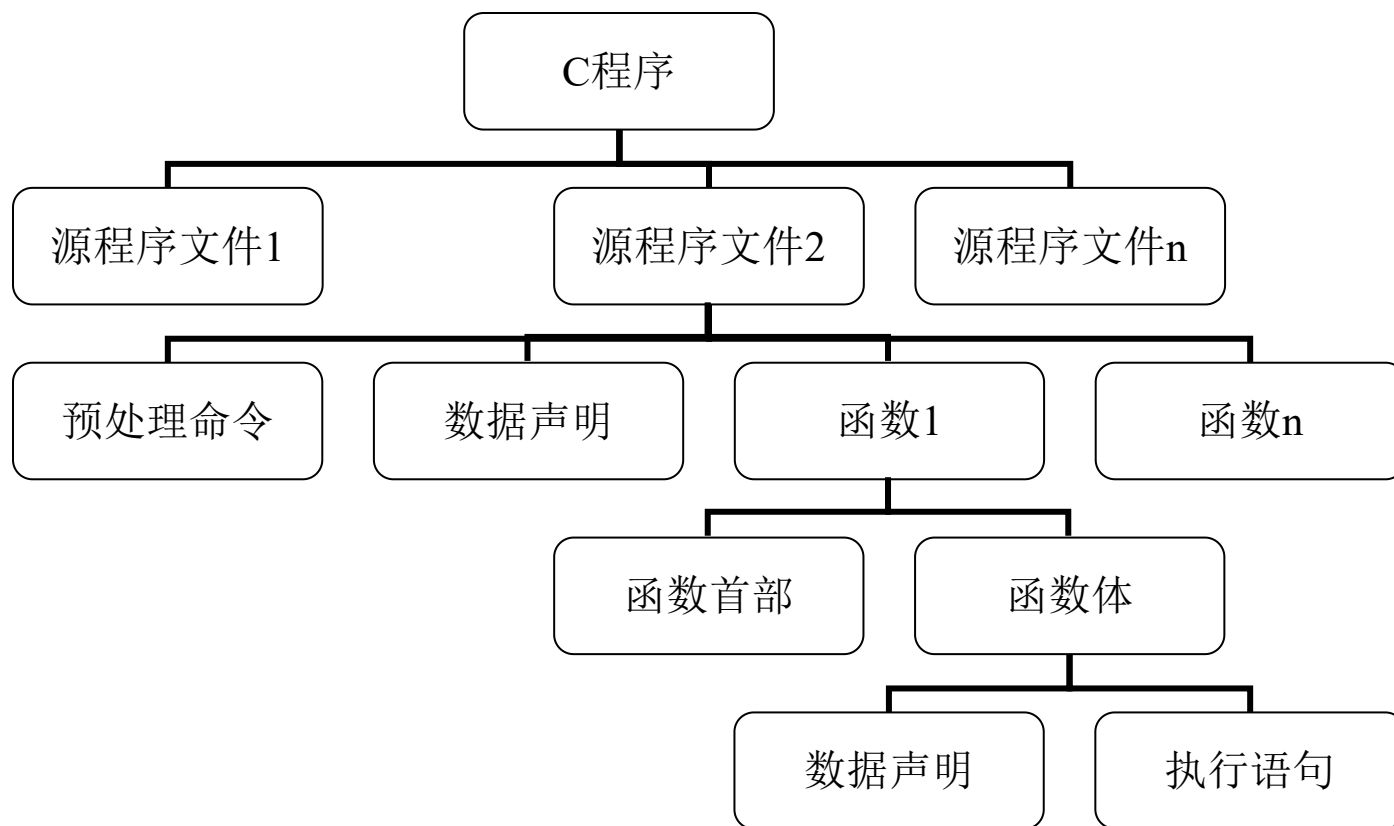
逗号表达式，常用于for()结构，也可用于表达式语句。

例： $a = 3 * 5, a * 4$;

分析：赋值运算符的优先级别高于逗号运算符，因此应先求解 $a = 3 * 5$ ，然后求解 $a * 4$ 得 60。

- ◆ 语句（statement）用于向计算机发出操作指令。
- ◆ 一条语句经编译后产生一条或多条机器指令。
- ◆ 一个实际的程序可能包括若干条语句。
- ◆ C语言规定了6类语句：

表达式语句、复合语句、选择语句、
循环语句、标记语句、跳转语句



一、表达式语句

表达式：算术表达式、关系表达式、逻辑表达式、赋值表达式。

表达式语句：在一个表达式末尾加上分号 “;”
(分号是C语言中语句的结束符)。

;
(空语句, 常用于if-else结构)

$i < j$;
(一个表达式语句, 但值被丢弃)

$i = i + 10 * j$;
(赋值表达式语句)

`printf("age=%d", getage());`
(函数调用表达式)

二、复合语句

复合语句是包含零个或多个语句的代码单元，使得一组语句成为一个整体，也被称为块。

在C语言中，复合语句由一个左花括号、可选语句段、一个右花括号组成。如：

```
{  
    i = 1;  
    j = 2 * i;  
}
```

◆计算机中的指令除了运算，还需要控制语句的执行序列，如：是非选择、多项选择、周而复始的循环、中断、意外终止等。

1 if() ~else 条件语句

2 for()~ 循环语句

3 while()~循环语句

4 do ~while();循环语句

5 continue 继续语句

6 break 间断语句

7 switch() 开关语句

8 return 返回语句



scanf and printf formats

%[flags][width][.precision][length]type

Type – usually for integers:

example

%d	Decimal (signed)	-12345
%i	scanf: Signed int , but allows octal, decimal, and hexadecimal input, depending on the prefix.	
%u	Decimal (unsigned)	53191
%o	O ctal	147707
%x %X	Hex a decimal	cfc7

Type – usually for floating points:

%f	Fixed-point notation	123000.00
%e %E	Exponential notation	1.23e+005
%g %G	%f or %e, whichever is shorter	

Type – usually for text:

%c	Print a C haracter	
%s	S tring	
%[] %[^]	scanf: Scans only the characters in the set. (%[^] excludes them instead)	%[aeiou] %[^12345]

Type – special:

%%	Single '%' character	
%n	Reads and prints nothing, but outputs the number of characters read/printed so far. (Argument must be an int*)	
%p	P ointer address	

Note

For printf, * can be used in the field width or precision (or both). In that case the function takes an additional int argument – preceding the argument to be formatted – to specify the width or precision. (takes 2 arguments if both are *, like %*.*f)

Flags (for printf only, except for the * flag)

%-4d	12	Left-justify the field instead of right.
%+d	+12	Always prepends the sign (+-).
% d (space)	12	Inserts a space if there's no sign.
%#o %#X	014 0xC	(For o, x, X) Precedes value with '0' or '0x'.
%#.0f %#.0e	12000034. 1.e+007	(For f, e) Prints the decimal point even if no digits follow.
%#.3g	1.00e+007	(For g) Keeps trailing zeros, along with decimal point.
%04d	0012	Pads the field with zeros instead of spaces.
%*c		scanf: Retrieves the data but discards it.

Field width

scanf: Maximum number of characters to be read.

printf: Minimum number of characters to be printed.

Precision (for printf only)

%.4d	0012	(For d, u, o, x, X) Minimum number of digits to be printed.
%.4f %.3e	12.3400 1.234e+001	(For f, e, E) Number of digits after the decimal point.
%.4g	12.34	(For g, G) Maximum number of significant digits.
%.4s	Prec	(For s) Maximum number of characters to be printed.

Length

%hd %hf	short (i.e. half length)
%ld %lf	long (For long double, use %Lf.)

文





引言

建筑物需要有良好的结构来抵御外界风险。

同样，一个程序也应该具有良好的结构，易于理解和维护。

结构化程序设计方法(Structured programming method)

- 自顶向下 (Top-down)
- 逐步求精 (Progressive refinement)
- 模块化 (Module)
- 结构化编码 (Structured coding: three basic structure)

伪代码表示的算法结构

伪代码是介于过程步骤的自然语言算法描述，和用实际编程语言描述过程之间的中间体。伪代码的基本结构类似于编程语言的控制结构，但比编程语言规范更自由，允许自然语言的步骤描述。

```
procedure max( $a_1, a_2, \dots, a_n$ : integers)
    max :=  $a_1$ 
    for  $i := 2$  to  $n$ 
        if  $max < a_i$  then  $max := a_i$ 
    return max {max is the largest element}
```

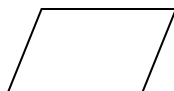
大括号{}在伪代码中表示注释，而用缩进表示嵌套的关系。

程序的结构描述—流程图

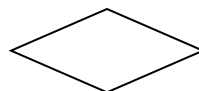
- 流程图的基本元素（ANSI规定）



起止框



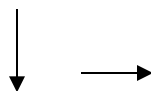
读入/出框



判断框



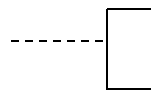
处理框



流程线



连接点



注释框

程序的结构描述—N-S流程图

- 基本结构

顺序结构

选择结构

当型循环结构

直到型循环结构

p、p1表示的是
判断条件；A、
B框是操作；

注意：A、B框可
以是一个简单
的操作（如读
入数据或打印
打印等），也
可以是三种基
本结构之一

三种控制结构的共同点

- 只有一个入口
- 只有一个出口
- 结构内的每一个部分都有机会被执行到
- 结构内不存在“死循环”（无终止的循环）

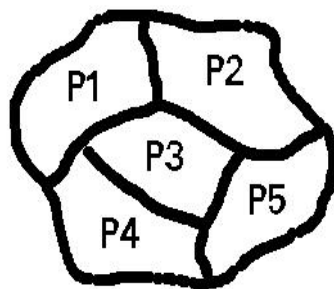
由这三种基本结构顺序组成的算法结构，可以解决任何复杂问题！

引入子程序的目的

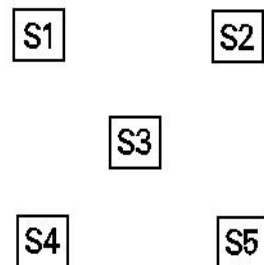
1. 程序“复用”，避免在程序中使用重复代码；
2. 结构化程序设计的需要：

自顶向下、逐步细化，将复杂问题分解为相对简单的子问题，这些子问题用子程序实现，从而提高主程序结构的清晰性和易读性。

3. 使程序的调试和维护变得更加容易。



需要通过软件解决的“问题”



软件的“解决方案”

四. 子程序设计原则

高内聚：功能相对独立和完整；

低耦合：与外界（调用者）的关系尽量松散，
不要太紧密，使其能方便地被重用；

需要合理地设计子程序参数和子程序执行的局部环境 来达到以上目标。

五. 子程序在C语言中的实现机制

C语言中的函数机制

函数定义的常见错误

- 常见的函数定义错误:

- (1) 把同一种类型的参数声明为类似于形式`f(float x, y)`, 而不是`f(float x, float y)`; //f是函数
- (2) 在函数内部把函数参数再次定义成局部变量是一种语法错误; 如:

```
int sum(int x, int y)
{
    int x, y; //与形参同名, 错误!
    return (x+y);
}
```

- 常见的函数调用错误: 1)实参个数和形参个数不一致; 2)实参类型和形参类型不兼容; 3)函数返回值类型和调用语句的赋值变量类型不兼容;

形式参数和实际参数

形式参数：函数原型中函数名后面括号中的变量名称为“形式参数”（简称“**形参**”）。

实际参数：函数调用语句中，函数名后面括号中的参数（可以是一个表达式）称为“实际参数”（简称“**实参**”）。

函数返回值：return语句中的表达式是函数带回的值（称**函数返回值**）。

返回值类型与函数类型不同的错误示例

```
# include <stdio.h>
int max(float x, float y);
int main(){
    float a,b;
    int c;
    scanf("%f%f", &a, &b);
    c = max(a,b);
    printf("Max is %d\n",c);
}
int max(float x, float y){
    float z;                                /* z为实型变量 */
    z = x>y ? x : y ;
    return (z);
}
```

函数调用的参数传递

子程序参数传递两种方式：**按值传递**和**按引用传递**。

按值传递：实参的值被复制并置入被调用子程序的形参中。此方式下不管在被调用子程序中怎样操作并改变形参的值，在主调程序中的实参的值都是安全的未发生变化的。

C语言中所有的调用都是**传值调用**。但是指针类型的使用模拟实现了数据引用传递的效果。

实参

实参

按值传递（将实参的值传递给形参）

形参

形参

按值传递

```
float square(float ); /*函数原型*/
```

函数原型的设计

(1) C语言的原则（先声明、后使用）；

(2) 函数原型的作用：函数原型是对被调用函数的**接口声明**，它告诉编译器函数返回的数据类型、函数所要接收的参数个数、参数类型和参数顺序，编译器用函数原型校验函数调用是否正确。

例： `float square(float ); /*函数原型*/`

(3) 函数原型一般格式：

返回值类型 函数名(数据类型[参数名1][, 数据类型[参数名2]...]); 注意：参数名可写可不写。

(4) 函数原型的**位置**:

- 1、可以放置在任何函数之外，出现在该函数原型之后的所有函数都可以调用对应的函数；
- 2、也可以放在某个函数中，此时只有该函数能够调用它；
- 3、从程序设计风格考虑，最好是将函数原型集中在一起，放在主函数之前。

函数的调用原理—栈区

- **栈区**：先进后出，后进先出。体现了递归函数的反序特性。
- 一片存放用户数据的内存空间；一端“封”了口（栈底），一端“开”着口(栈顶)；
- **保存数据**：数据只能从顶部（**栈顶**）进入；
- **取数据**：栈顶的数据先被取出，**栈底**的数据最后被取出。即数据是“**先进后出**”。数据的进入和退出均在栈顶进行。
- **读数据**：只能读取
栈顶的数据

栈顶

栈底



函数调用原理：函数栈的函数活动记录

- 为方便描述函数调用后产生的实例，引入一个术语：“**函数的活动记录**”。函数的活动记录是一段在**栈区**分配的连续的内存存储区，用以存放函数一次执行所需的数据。
- 函数调用的启动，要在**函数栈区的栈顶**创建一个函数活动记录。函数返回，要从函数栈顶将函数的活动记录释放。

被调用函数中
的数据

现场信息，用
于调用结束能
正确返回

- 数据结构的三要素：
 - 数据模型：数据类型、数据个数、数据之间的关系。
 - 整数、小数、字符编码；
 - 线性表、树、图。
 - 数据的存储方式
 - 连续存储，随机存储；
 - 数据的操作算法
 - 查找、增加、修改、删除...

- **线性表**是具有相同数据类型的 $n(n \geq 0)$ 个数据元素的有限序列，通常记为： $(a_1, a_2, \dots, a_{i-1}, a_i, a_{i+1}, \dots, a_n)$ 其中 n 为表长， $n=0$ 时称为空表。
线性表数据元素之间为线性关系，即任意两个元素之间都可比较大小。
- **线性表有两种存储结构：连续（顺序表）、随机（链表）。**
- 线性表的算法可归纳一些基本操作，如：
 - 创建与更新：增加、删除表中的元素；
 - 检索：查找表中的某个元素；
 - 排序：按照元素值的某种排序规则对线性表中的元素进行重新排列。

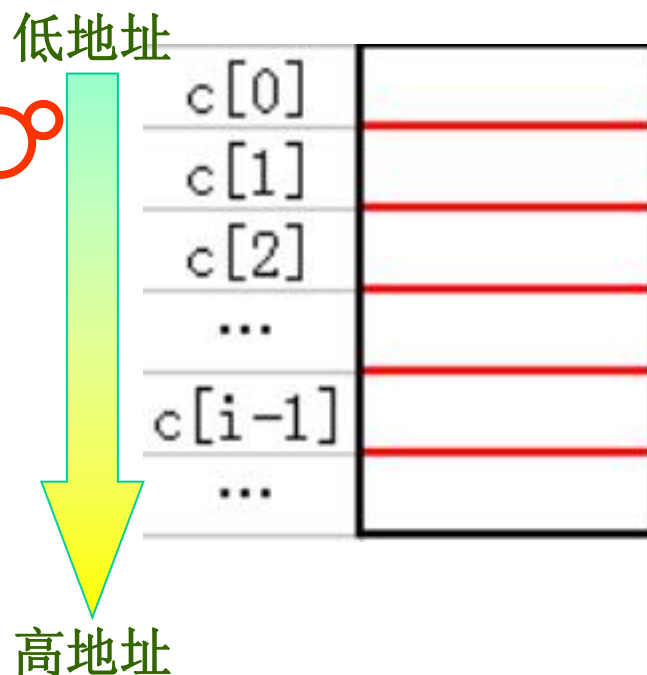
数组是连续存储的线性表

- 数组必须一次性整体定义，预先确定数据类型和**元素最大个数**，因为要占用一组**连续**的存储单元。

定义 元素类型名 数组名[**整型常量**表达式];

例： int c[100];

每个数据元素
占用的字节数，
就是**基类型**的
字节数



- 数组必须逐个元素访问（操作）
 - 注意：第一个元素的序号为0。
- 数组元素操作，按**数组名[下标]**读取或赋值。
- C语言中可利用声明语句对数组元素的值进行初始化：
元素类型名 数组名[整型表达式]={值，值……值}；
- 数组名作为函数的参数时，传递的是数组的首地址（第一个元素的地址）。使得被调用函数能够访问、操作原数组！

- 变量的访问：间接访问—在变量作用域之外,可以通过指针(pointer)来访问。
- C 程序的**指针变量**，用来存放所指变量的内存起始地址。
- 指针的类型前缀是*，语法：**数据类型** * 指针变量名；
- 指针的间接访问运算符是*，表达式 (*指针变量名)代表指针指向的数据变量。

/*在子函数中交换主调函数的两个变量*/

```
void swap(int * p1, int * p2){
```

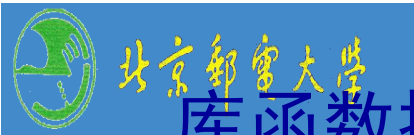
```
    int temp;
```

```
    temp=*p1;
```

```
    *p1=*p2;
```

```
    *p2=temp;
```

```
}
```



库函数提供动态地开辟和释放存储单元的

有关函数：

(1) malloc函数

其函数原型为`void *malloc(unsigned int size);`其作用是在内存的动态存储区中分配一个长度为size的连续空间。此函数的值（即“返回值”）是一个指向分配域起始地址的指针（类型为void）。如果此函数未能成功地执行（例如内存空间不足），则返回空指针(NULL)。

(2) free函数

其函数原型为`void free (void *p)` ;其作用是释放由 p 指向的内存区，使这部分内存区能被其他变量使用。



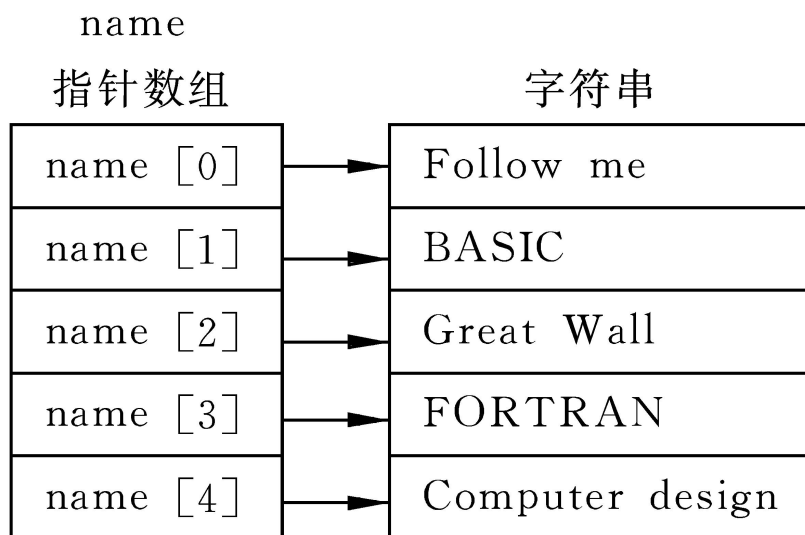
字符串

Follow me
BASIC
Great Wall
FORTRAN
Computer design

(a)

F	o	l	l	o	w		m	e	\0						
B	A	S	I	C	\0										
G	r	e	a	t		W	a	l	l	\0					
F	O	R	T	R	A	N	\0								
C	o	m	p	u	t	e	r		d	e	s	i	g	n	\0

(b)



(c)

```
void strSort(char * name[], int n){
    char * pt;
    int i, j, min;
    /*使用选择排序，外循环每循环一次,确定a[i]的值*/
    for(i=0; i<n-1; i++){
        min=i;
        for(j=i+1; j<n; j++)
            if(strcmp(name[min], name[j])>0)
                min=j;
        if(min!=i){
            pt= name[min];
            name[min]= name[i];
            name[i]=pt;
        }
    }
}
```

